Laboratorio 5: Seguridad en APIs REST con JWT y OAuth 2.0

Juan Miguel Rojas Chaparro, Cristian David Silva Perilla, David Eduardo Salamanca, Felipe Eduardo Calvache
Escuela Colombiana de Ingeniería Julio Garavito
Bogotá, Colombia

***Resumen*—En este laboratorio se implementó seguridad para una API REST de Blueprints usando Spring Security, OAuth 2.0 Resource Server y JSON Web Tokens (JWT) firmados con RS256. Se configuró autenticación mediante un endpoint de login, autorización por scopes (`blueprints.read` y `blueprints.write`) y validación funcional del flujo completo mediante pruebas manuales en Swagger UI.**

***Index Terms*—JWT, OAuth 2.0, Spring Security, Resource Server, RS256, Swagger.**

I. Introducción

Las APIs REST expuestas en red requieren controles de acceso que aseguren autenticación y autorización por rol/-permiso. Este laboratorio extiende el API Blueprints para que opere como *Resource Server*, exigiendo token Bearer en endpoints protegidos y aplicando control granular por alcances (*scopes*).

II. Objetivos

- Implementar seguridad REST con OAuth2 Resource Server.
- Configurar emisión y validación de JWT.
- Proteger endpoints por scopes: `blueprints.read`, `blueprints.write`.
- Documentar y validar el flujo desde Swagger/OpenAPI.

III. Arquitectura de Seguridad

III-A. Modelo de autenticación

Se implementó un endpoint didáctico `/auth/login` que valida credenciales contra usuarios en memoria y retorna un `access_token` JWT con expiración configurable.

III-B. Modelo de autorización por scopes

La autorización se configuró por verbo HTTP y por método de controlador:

- GET sobre `/api/**`: requiere `blueprints.read`
- POST y PUT sobre `/api/**`: requieren `blueprints.write`

III-C. Asignación de scopes por usuario

La asignación se realiza durante login:

```
String scope = switch (req.username()) {
    case "student" -> "blueprints.read";
    case "assistant" -> "blueprints.read_blueprints.write";
    default -> "";
};
```

Listing 1: Asignación de scope por usuario autenticado.

IV. Implementación Técnica

IV-A. Dependencias usadas

- `spring-boot-starter-security`
- `spring-boot-starter-oauth2-resource-server`
- `nimbus-jose-jwt`
- `springdoc-openapi-starter-webmvc-ui`

IV-B. Configuración de seguridad

Se definió un `SecurityFilterChain` con endpoints públicos y protegidos:

```
.requestMatchers("/auth/login").permitAll()
.requestMatchers("/v3/api-docs/**", "/swagger-ui/**", "/
    swagger-ui.html").permitAll()
.requestMatchers(HttpMethod.GET, "/api/**").hasAuthority("
    SCOPE_blueprints.read")
.requestMatchers(HttpMethod.POST, "/api/**").hasAuthority("
    SCOPE_blueprints.write")
.requestMatchers(HttpMethod.PUT, "/api/**").hasAuthority("
    SCOPE_blueprints.write")
```

Listing 2: Reglas de autorización en SecurityConfig.

IV-C. JWT RS256

La aplicación genera llaves RSA en memoria:

- Clave privada para firmar tokens.
- Clave pública para validarlos.

Esto garantiza integridad del token y separación entre emisión y verificación.

V. Validación Funcional en Swagger (Evidencias)

V-A. Preparación del entorno

1. Ejecutar: `mvn -q -DskipTests spring-boot:run`
2. Abrir: http://localhost:8080/swagger-ui/index.html

V-B. Prueba 1: acceso sin token

Objetivo: verificar protección base.
Acción: ejecutar `GET /api/blueprints` sin autenticar.
Resultado esperado: `401 Unauthorized`.

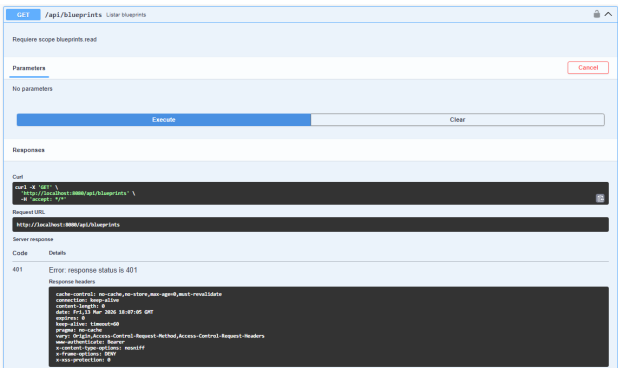

Figura 1: Evidencia: GET sin token retorna 401.

V-C. *Prueba 2: login de estudiante*

Acción: `POST /auth/login` con:

```
{
  "username": "student",
  "password": "student123"
}
```

Listing 3: Credenciales usuario student.

Resultado esperado: respuesta con `access_token`, `token_type`, `expires_in`.

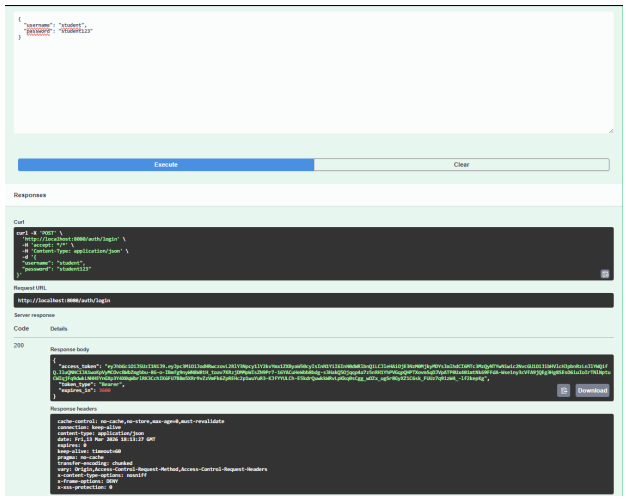

Figura 2: Evidencia: login de student y token emitido.

V-D. *Prueba 3: student con scope de lectura*

- Authorize con token de `student`.
- Ejecutar `GET /api/blueprints`: esperado `200 OK`.
- Ejecutar `POST /api/blueprints`: esperado `403 Forbidden`.

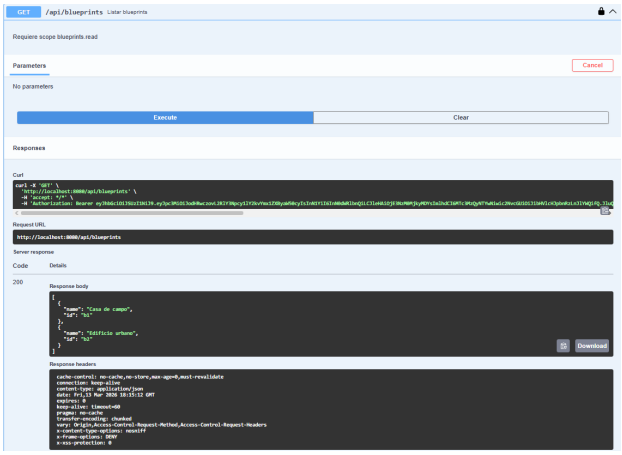

Figura 3: Evidencia: student puede consultar (GET=200).

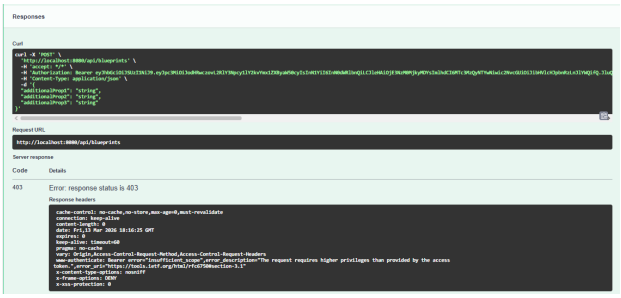

Figura 4: Evidencia: student no puede crear (POST=403).

V-E. *Prueba 4: login de assistant*

Acción: `POST /auth/login` con:

```
{
  "username": "assistant",
  "password": "assistant123"
}
```

Listing 4: Credenciales usuario assistant.

Resultado esperado: token con permisos de lectura y escritura.

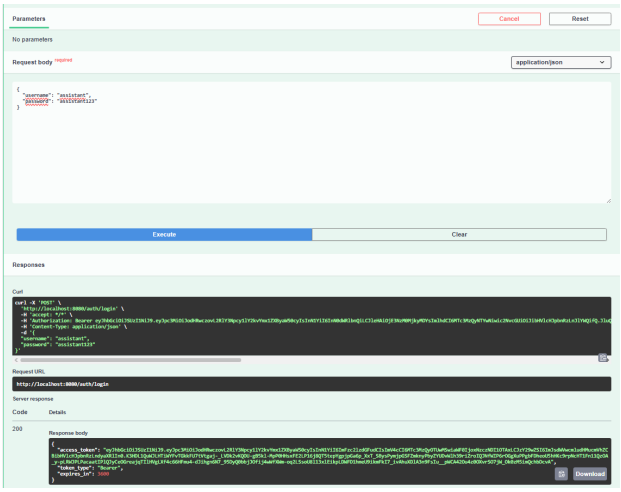

Figura 5: Evidencia: login de assistant y token emitido.

V-F. *Prueba 5: assistant con scope de escritura*

- Authorize con token de `assistant`.
- Ejecutar `POST /api/blueprints`: esperado `200 OK`.
- Ejecutar `PUT /api/blueprints/.../points`: esperado `200 OK`.

```
{
  "x": 1,
  "y": 2
}
```

Listing 5: Body usado en PUT de puntos.

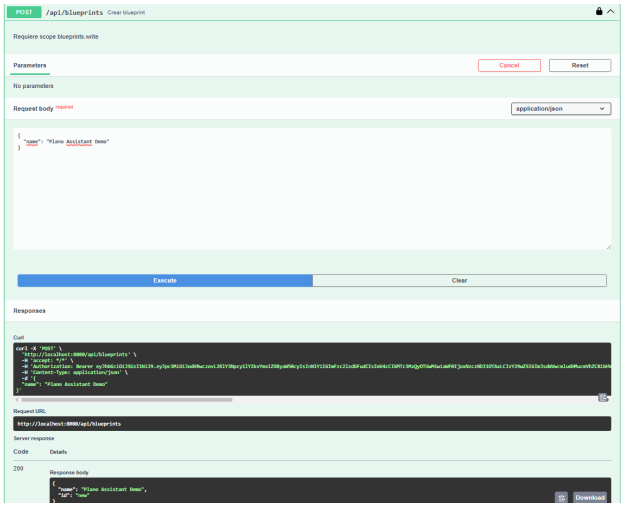

Figura 6: Evidencia: assistant puede crear (POST=200).

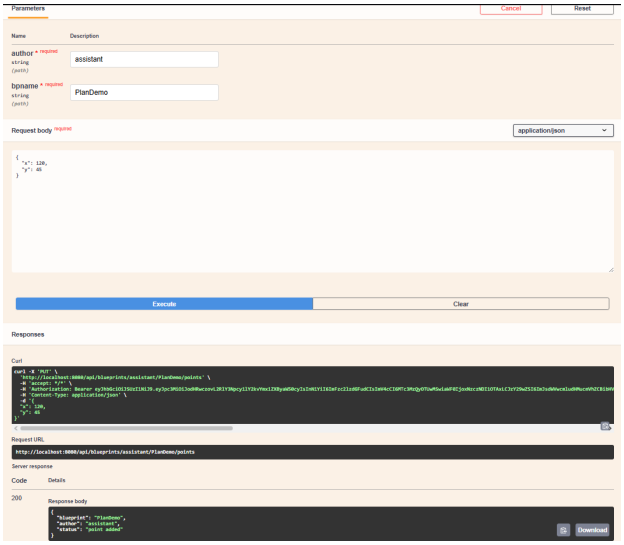

Figura 7: Evidencia: assistant puede modificar (PUT=200).

VI. Resumen de resultados

Cuadro I: Matriz de validación de scopes

Escenario	Endpoint	Resultado
Sin token	GET /api/blueprints	401
student	GET /api/blueprints	200
student	POST /api/blueprints	403
assistant	POST /api/blueprints	200
assistant	PUT /api/blueprints/author/bpname/points	200

VII. Conclusiones

1. La API quedó correctamente protegida como Resource Server con JWT.
2. El uso de scopes permitió control granular por operación (`read` vs `write`).
3. Swagger permitió validar de forma trazable y reproducible el flujo de seguridad completo.
4. Los resultados obtenidos son consistentes con los objetivos del laboratorio y evidencian cumplimiento funcional.

Referencias

[1] Spring Security Reference, OAuth 2.0 Resource Server, 2024. [Online]. Available: https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/index.html

[2] JWT.io, JSON Web Tokens Introduction, 2024. [Online]. Available: https://jwt.io/introduction

[3] Springdoc OpenAPI, 2024. [Online]. Available: https://springdoc.org/